Projektuppgift

Kursnamn: DT224G

Titel: Kaffe Grus
Eventuell underrubrik: En modern, responsiv och tillgänglig webbplats för en lokal kaffeteria

Förnamn Efternamn: Paul Yashouh



Mittuniversitetet
MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. **Campus Sundsvall** Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00. Fax: +46 (0)771 97 50 01.

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi

Författare: Paul Yashouh, paya2600@student.miun.se
Utbildningsprogram: Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: HT, 2026

1 Sammanfattning

Denna rapport beskriver hur en modern och mobilanpassad webbplats utvecklades för det fiktiva kaféet Kaffe Grus. Kaféet målgrupp är studenter och personer som arbetar på distans. De behöver en digital plattform för att enkelt kunna se kaféets meny, priser och öppettider innan de gör ett besök.

Projektet började med planering och design i verktyget Figma. Webbplatsen byggdes sedan från grunden med fokus på att fungera bra på mobiltelefoner samt datorer. Sidorna använder HTML, CSS och JavaScript för att hämta produkter live från ett externt API. Eftersom produkterna från API-länken saknade priser skapades kod som automatiskt ger produkterna rimliga priser på skärmen. På kontaktsidan skapades ett fungerande formulär och interaktiv Google-karta. För att säkra kvaliteten testades webbplatsen genom W3C-validering och i olika webbläsare.

Resultatet är en stabil och lättanvänd webbplats som uppfyller nästan alla designkrav. På grund av tidsbrist hann dock inte funktionerna för produktfiltrering, prenumeration och en overlay med extra produktinformation färdigställas. Slutsatsen är att grunden för applikationen är stark och fungerar bra för målgruppen, men att de saknade delarna bör byggas vidare på i framtiden.

Länk till den färdiga applikationen finns här:
https://projektuppgift.vercel.app/

2 Innehållsförteckning

1 Sammanfattning 3
2 Innehållsförteckning 4
3 Introduktion 5
3.1 Sammanhang och målgrupp 5
3.2 Kommunikativt Syfte 5
3.3 Problemmotivering 5
3.4 Problemformulering 6
4 Teori / Bakgrund 7
4.1 Centrala webbtekniker och verktyg 7
4.2 Responsiv Webb design och Mobile-First 7
4.3 Asynkron JavaScript och REST API 7
4.4 Webbtillgänglighet 8
5 Metod 9
5.1 Planering och Design i Figma [5] 9
5.2 Systemutveckling och versionshantering 9
5.3 Hantering av felaktig commit i fel Git-gren 9
5.4 Testning 10
6 Konstruktion 11
6.1 Semantisk HTML-struktur och återanvändbarhet 11
Problemet åtgärdaades grnom att en anpassad favicon-ikon lades till på samtliga HTML-sido i dokumenthuvud via elementet: 11
6.2 Responsiv CSS-arkitektur och Mobile-first 11
6.3 JavaScript-arkitektur och API-integration 12
7 Resultat 14
7.1 Uppfyllnad av projektmål och funktionalitet 14
7.2 Avgränsningar och saknad funktionalitet 14
7.3 Resultat från tester 14
8 Resultat 16
9 Referenser 17

Introduktion

2.1 Sammanhang och målgrupp

I ett digitalt samhälle har arbetsmönstren förändrats markant, vilket har lett till en kraftig ökning av distansarbete., frilansar och studerande som söker flexibla arbetsmiljöer utanför det traditionella kontoret. Kaffer har i stor utsträckning transformerats till tillfälliga arbetsplatser, så kallade ”tredjeplatser”.

Webbplatsen Kaffe Grus är utvecklad för ett modernt kafé på Holmgatan Sundsvall med en filialinriktning i Sundsvall. Verksamhetens unika profil kombinerar försäljning av specialkaffe med att erbjuda en optimerad arbetsmiljö.

Den primära målgruppen är onlinearbetare och studenter som kräver specifik information gällande kaféets facilitet såsom tillgång till snabbt, kostnadsfritt Wi-Fi, eluttag och studieplatser i kombination med ett kvalitativt dryckesutbud.

2.2 Kommunikativt Syfte

Det huvudsakliga kommunikativa syftet med webbplatsen är att fungera som en interaktiv, digital knutpunkt som sänker tröskeln för målgruppen att besöka kaféet fysiskt.

Genom applikationen ska användaren sömlöst kunna navigera mellan digitala sektioner för att utforska produktutbudet, granska priser och ingredienser för både varma och kalla drycker, lokalisera verksamheten geografiskt via en integrerad kartlösning sam hitta relevanta kontaktvägar och öppettider.

2.3 Problemmotivering

Små och nystartade verksamheter inom kafébranchen möter ofta utmaningen arr synas digitalt samt att presentera information på ett tillgängligt sätt. Traditionella statiska PDF-Menyer skapar frustration hos användare som snabbt behöver fatta beslut basera på utbud och priser. För online-arbetare är tillförlitlighet och omedelbar tillgång till information avgörande innan de väljer att etablera sig på en fysisk plats för en arbetsdag.

Ett annat problem för verksamheten är den administrativa belastningen på personalen. När kunder tvingas ringa till kaféet för att ställa repetitiva frågor om vad som erbjuds eller efterfråga basinformation, skapar det ett tidskrävande moment som avbryter kaffearbetarenas ordinarie arbetsflöde i den fysiska serveringen. Det finns därför ett stort behov av en sammanhållen, modern problem plattform som effektiviserar kommunikationen och avlastar personalen genom att göra information direkt tillgänglig.

2.4 Problemformulering

Projektets ska lösa verksamheten utmaningar genom att utveckla en modern och helt mobilanpassad webbplats. För att uppfylla den huvudsakliga behoven hos Kaffe Grus ska den färdiga lösningen åstadkomma följande konkreta mål:

- Centraliserad och komplett information: Samla hela kaféets utbud av varma och kalla drycker med fullständiga information, priser, kontaktuppgifter och öppettider på ett och samma ställe så att kunderna slipper ringa för att ställa vanliga frågor.

- Enkel lokalisering: Göra det smidigt för besökare att hitta till den fysiska kaffebaren på Holmgatan med hjälp av interaktiv, inbäddad karta direkt i gränssnittet.

- Interaktiv kontakt: Tillhandahålla ett lättanvänt och säkert kontaktformulär där besökare snabbt kan skicka meddelande till verksamheten utan krångel.

- Fullständig flexibilitet: Garantera god visuell tillgänglighet och ett gränssnitt som anpassar sig helt perfekt oavsett om målgruppen använder en mobiltelefon på språng eller en dator under ett arbetspass.

3 Teori / Bakgrund

3.1 Centrala webbtekniker och verktyg

För att bygga denna webbplats har en kombination av olika tekniker och verktyg använts för att skapa en responsiv. Interaktiv och modern applikation:

- HTML (HyperText Markup Language): Används som grundstruktur för att bestämma vilket innehåll som ska finnas på webbplatsen, som text, bilder och formulär.

- CSS (Cascading Style Sheet): Används för att designa utseendet, färgerna och layouten samt att göra webbplatsen responsiv så att den passar olika skärmstorlek.

- JavaScript: Används för att göra webbplatsen interaktiv för användaren, hantera logik samt sköta kommunikationen med externa datakälla.

- REST API och JSON: Används som extern datakälla för att hämta produktdata i realtid till menyn.

- **Versionshantering (Git och GitHub): Git** används lokalt på datorn under hela utvecklingsprocessen för att spara kodens historik och hantera olika versioner av projektet. Koden laddas sedan upp till den molnbaserade plattformen **GitHub**, som fungerar som ett centralt arkiv för att lagra, säkerhetskopiera och organisera projektets kodbas.

Dessa tekniker samverkar genom att HTML bygger strukturen, CSS klär den i rätt design och JavaScript fungerar som hjärnan som hämtar den från API:et och gör sidan levande.

3.2 Responsiv Webb design och Mobile-First

Responsiv webbdesign innebär att en webbplatslayout och gränssnitt dynamiskt anpassar sig efter användarens skärmstorlek, upplösning och orientering. För att motverka bristfällig funktionalitet på mindre skärmar tillämpas strategin mobile-First, vilket innebär att design och utvecklingsprocessen utgår från den mobila enheten för att sedan expandera layouten gradvis för större skärmar via CSS media queries.

Genom att prioritera den mobila upplevelsen tvingas utvecklaren att skala bort överflödigt innehåll och fokusera på kärnfunktionalitet och prestanda. Inom modern CSS uppnås denna flexibilitet effektivt ned layoutmodulerna Flexbox, optimerad för endimensionella flöden och linjeringar, samt CSS Grid, designat för mer komplexa, tvådimensionella rutnätslayout.

3.3 Asynkron JavaScript och REST API

För att dynamiska webbapplikationer som inte kräver att hela webbsidan laddas om vid varje interaktion används asynkron programmering. Genom att skicka asynkront

anrop i bakgrunden kan applikationen förbli responsiv gentemot användaren samtidigt som data hämtas. Data överförs vanligtvis i JSON format vilket är ett lättviktigt och textbaserat format som enkelt kan sparas i JavaScript. I modern utveckling används API:et fetch() [1] tillsammans med async/await [1] syntaxen för att hantera dessa nätverksanrop på ett läsbart och strukturerat sätt.

- Användandet av async/await [1] löser två kritiska problem inom asynkron programmering: Undvikande av Callback Hell: Det förhindra den djupt nästlade och svårlästa kodstruktur

- Sekventiell exekvering: Det gör det möjligt att pausa exekveringen in funktion på ett kontrollerat sätt för att vänta tills nödvändiga data har hämtats fullständigt, innan applikationen går vidare med att utföra nästa logiska steg.

3.4 Webbtillgänglighet

I detta projekt har W3C:s officiella valideringsverktyg använts systematiskt för att kvalitetssäkra att både HTML-strukturen och CSS-stilmallarna följer gällande webstandarder.

För att garantera god visuell tillgänglighet och tillräckliga färgkontraster användes verktyget Coolors [2]under planeringsfasen. Med hjälp av Coolors [3]valdes en färgpalett dominerad av kaffebruna toner som både reflekterar kaféets grafiska identitet och säkrar läsbarheten mot bakgrunden. Som komplement till den grafiska profilen skapades även kaféets logotyp programmatiskt i designverktyget Canva [4].

4 Metod

4.1 Planering och Design i Figma

Projektet inleddes med en planeringsfas där webbplatsens struktur, funktionalitet och användarbehov definierades. Fokus lades på att skapa ett lättnavigerat gränssnitt, tydlig prissättning och en enkel geografisk lokalisering för målgruppen.

För att säkerställa att webbplatsen blev användarvänlig och att designen var genomtänkt innan kodningen påbörjades, användes designverktyget Figma [5]. Valet föll på Figma eftersom det gör det möjligt att arbeta iterativt och upptäcka designfel tidigt, vilket sparar tid under utvecklingen. Processen delades upp i tre steg:

1. Wireframe(lo-fi) EN lågupplöst strukturell skiss skapades för att bestämma layoutens grundarkitektur och placering av element helt utan färger eller typsnitt.

2. Mockup(hi-fi): EN högupplöst visuell design utvecklades där typografi, grafiska komponenter samt en färgpalett från Coolors [2] applicerades.

3. Prototyp: En interaktiv och klickbar modell skapades för att simulera det faktiska användarflödet och testa navigationen innan kodningsfasen påbörjades.

Det färdiga skisserna exporterades i PDF format och arkiverades i design mappen för att fungera som en visuell specifikation utvecklingen.

4.2 Systemutveckling och versionshantering

Utvecklingen genomfördes i kodredigeraren VS Code [6]. För att garantera en stabil källkod och en spårbar utvecklingshistorik tillämpades en strukturerad förgreningsteknik i Git med följande struktur:

- Main-gren (main): Fungerar som produktionsmiljö och innehåller enbart fullständigt testad och stabil kod.

- Development-gren(dev). Skapades utifrån (main) och agerade som den centrala integrationsgrenen under utvecklingsfasen.

- Funktionsgrenar/Featuregrenar: För varje specifik JavaScript funktion eller gränssnitt skapades en isolerad gren utifrån (dev).

4.3 Hantering av felaktig commit i fel Git-gren

Under utvecklingen råkade jag skriva funktionskod direkt i integrationsgrenen dev istället för i en isolerad funktionsgren, och hann även göra en commit på ändringarna där. Eftersom dev-gren ska hållas ren från direktkodning behövde jag flytta koden i efterhand.

Jag löste problemet genom att göra en git reset –hard [7] på dev-grenen för att återställa den till sitt ursprungliga läge. Eftersom koden då tillfälligt försvann använde jag kommandot git reflog [8]för attt leta upp och kopiera den unika commit-hashen för mina ändringar. Därefter checkade jag ut till den rätt funktionsgrenen(branch) och körde kommandot git cherry-pick [commit-hash] [9] för att plocka över ändringarna till rätt ställe.

När funktion var färdigutvecklad och lokalt verifierad integrerades koden tillbaka till dev-grenen. Detta genomfördes via kommandot git merge –no-ff. vilket tvingade Git att alltid skapa ett unikt sammanslagnings-commit. Genom denna metod förhindras en så kallad fast-forwardsammanslagning, vilket säkerställde att projektets historiska gren-struktur och de enskilda funktionernas utvecklingsspår förblev intakta och spårbara i versionshistoriken.

4.4 Testning

Kvalitetssäkringen av applikationen utfördes kontinuerligt genom tre huvudsakliga testmetoder.

- JavaScript-testning: All asynkron logik, prissättning och formulärvalidering verifierades löpande genom exekvering och felsökning direkt i webbläsarens inbyggda utvecklarverktyg (Console).

- W3C-kodvalidering: HTML-strukturen kontrollerades i Nu HTML Checker och CSS-Stilmallarna i W3C [10] CSS Validator för att garantera att källkoden strikt följer officiella webbstandarder.

- **Kompatibilitet och responsivitet:** Layouten testades systematiskt i Google Chrome och Mozilla Firefox utifrån en *Mobile-first*-princip, där gränssnittets Flexbox - och Grid-system utvärderades från mobila skärmstorlekar upp till stationära bildskärmar.

5 Konstruktion

5.1 Semantisk HTML-struktur och återanvändbarhet

Webbplatsen är uppbyggd med semantiska HTML5-element [11] (<header>, <nav>, <main>, <section>, <address>, <footer>) för att säkerställa hög tillgänglighet och sökmotoroptimering (SEO). Ett enhetligt sidhuvud och sidfot återanvänds på samtliga sidor. Sidhuvudet integrerar kaféets SVG-logotyp.

För att stärka kaféets digitala identitet och eliminera felmeddelande under utvecklingen upptäcktes ett 404-fel (Not Found) i webbläsaren konsol gällande en saknad standarikon (Figur 1).



Figur 1 Dekterat 404-fel i webbläsaren utvecklarverktyg (Console)för en saknad favicon

Problemet åtgärdades genom att en anpassad favicon-ikon lades till på samtliga HTML-sidor i dokumenthuvud via elementet:

```
 9
10     <link rel="shortcut icon" type="image/x-icon" href="assets/images/favicon.svg">
11
```

Detta resulterade i en helt ren och professionell produktionsmiljö utan konsolfel.

5.2 Responsiv CSS-arkitektur och Mobile-first

Stilmallarna i css/styles.css följer principen *Mobile-first*. Alla globala basstilar definieras utanför Media Queries för att minimera renderingsbördan på mobila enheter.

- **Custom Properties och typografi:** CSS-variabler (:root) används för färgpaletten (kaffebruna toner hämtade från Coolors). Som visas i (Figur 2) Typografin använder den moderna CSS-funktionen clamp() [11] för att uppnå en helt flytande, skalbar textstorlek utan tvära brytpunkter.

```
399  /* MENY KORT SEKTION SIDA*/
400  .meny h1 {
401    font-size: clamp(3.2rem, 2.5vw, 4.8rem);
402    color: var(--primar-farg);
403  }
```

Figur 2Flexibel typografisk skalning i CSS med funktionen clam()

- **Flexbox och navigationsanimation:** Flexbox hanterar elementjusteringar. Mobila navigeringen döljs utanför skärmkanten med transform: translateX(-100%) och glider mjukt in via klassen ”.oppen”. Hamburgarknappen animeras samtidigt till ett kryss (X) genom att rotera de yttre strecken 45 grader via :nth-child-selektorer.

- **Skalbarhet och responsiva brytpunkter:** Vid brytpunkten 768px transformeras sidfoten och kontaktheron till horisontella flöden (flex-direction: row). Menysidan omstruktureras till ett tvådimensionellt *CSS Grid* med repeat(auto-fit, minmax(280px, 1fr)), vilket skapar ett flytande produktutbud utan horisontell scroll. Egenskapen align-items: stretch garanterar en symmetrisk höjd på alla produktkort.

5.3 JavaScript-arkitektur och API-integration

Webbplatsens logik drivs i strikt läge ('use strict') via js/main.js. All DOM-manipulation initieras först efter händelsen DOMContentLoaded. Villkorsstyrda block säkerställer att API-logiken exekveras exklusivt på menysidan för att spara nätverksresurser.

- **Dynamisk navigering:** Navigeringsmenyn genereras programmatiskt genom att mappa en array med sidobjekt. Den aktuella sidan identifieras via window.location.pathname för att automatiskt tilldela stilklassen ”.aktiv”.
- **Asynkron inläsning och datatransformering:** Produktdata hämtas asynkront från *SampleAPIs* med fetch() [1] och async/await [1] för att undvika *callback hell*. Datamängden begränsas med slice(0, DATA_LIMIT) för att undvika och hämta all data. Eftersom prisuppgifter saknades i API-datan skapades hjälpfunktionen adderPris(), som visas i (Figur 3) transformerar arrayen via .map() [12] metoden och injicerar slumpmässiga priser mellan 90 och 120 kronor med Math.floor(). I denna funktion används **spread-operatorn** (…item) [13] för att kopiera över alla befintliga egenskaper från API objektet som och lägger till det nya slumpade pris utan att skriva över originaldatan.

```
84  // Helper för SlumpProdukt Pris (randomPrice)
85  function adderPris(itemArray) {
86    return itemArray.map((item) => {
87      const randomPrice = Math.floor(Math.random() * (120 - 90 + 1) + 90)
88      return {
89        ...item,
90        pris: randomPrice,
91      }
92    })
93  }
```

Figur 3 Hjälpfunktionen adderPris som transformera produkter och injicera slumpmässig priser via spridningsoperatorn (...tem)

- **Defensiv rendering och felhantering:** Funktionen visaData() använder defensiva kontrollsteg för att säkerställa ett robust system. Eventuella testdata (BRPROBE) exkluderas, långa texter trunkeras med substring(), och trasiga bildlänkar ersättas automatiskt via attributet onerror som en fallback bild. Om ett nätverksfel uppstår fångas det upp i en catch-block och ersätter menyn med ett tydligt felmeddelande i röd text.

- **Programmatisk formulärgenerering och validering:** Kontaktformuläret genereras helt programmatiskt i DOM-trädet med document.createElement(). Vid inskickning förhindras sidomladdning med e.preventDefault() [14]. Valideringsfunktionen kontrollerar att fälten inte är tomma med. trim() [15] metoden. Eventuella felmeddelanden samlas i en array och skrivs ut dynamiskt, medan ett godkänt formulär nollställs omedelbart via formSektion.reset().

6 Resultat

6.1 Uppfyllnad av projektmål och funktionalitet

Resultatet av arbetet är en fungerande applikation som i stora drag har utgått från den framtagna Figma-mockupen, där majoriteten av de primära funktionerna implementerats framgångsrikt.

Webbplatsen uppfyller samtliga uppställda krav gällande struktur, visuell hierarki och gränssnittsdesign. Genom att tillämpa en strikt *Mobile-first*-metodik flyter gränssnittet sömlöst mellan mobila enheter och stationära datorer. Den semantiska strukturen säkerställer att skärmläsare och andra hjälpmedel logiskt kan tolka sidans innehåll.

Nätverksanropen mot de externa API-ändpunkterna exekveras asynkront utan fördröjningar i användargränssnittet. Varma och kalla drycker hämtas och renderas dynamiskt som komponenter i sina respektive sektioner på menysidan.

Det egenutvecklade randomiseringsskriptet genererar stabila och trovärdiga prisuppgifter för de produkter som saknade detta i rådatan, vilket skapar ett sammanhängande och produktionsredo intryck för slutanvändaren. Kontaktsidan uppvisar en full klient validerat formulär tillsammans med en interaktiv Google Maps-karta via ett <iframe> [16]-element, vilket uppfyller målgruppens krav på enkel geografisk lokalisering.

6.2 Avgränsningar och saknad funktionalitet

På grund av projektets givna tidsram fanns det dock inte utrymme att färdigställa samtliga visuella och interaktiva element från mockupen. De delar som inte hann uppfyllas och implementeras är:

- Filtreringsfunktionen för produkterna
- Prenumerationssystemet
- Overlay-fönstret(modalen) som enligt skissen skulle öppnas för att visa mer information om varje produkt.

Dessa funktioner har därför exkluderats från det slutgiltiga resultatet och lämnas som förslag till framtida vidareutveckling av systemet.

6.3 Resultat från tester

För att kontrollera att webbplatsen fungerar som tänkt genomfördes de tester som planerades i metodkapitlet. Resultatet visar att applikationen är stabil, med ett par mindre avvikelser vid kodvalideringen:

Test	Förväntan	Resultat
Öppna och stäng mobilmenyn	Menyn öppnas och stängs mjukt vid klick på hamburgarknappen	Godkänt
Skicka tomt kontaktformulär	Inskickning stoppas och tydliga felmeddelanden visas för användaren	Godkänt
Klientvalidering av formulär	Formuläret nollställs korrekt när giltiga data skickas in	Godkänt
Dynamisk prissättning	Skriptet tilldelar alla produkter ett logiskt pris mellan 90 och 120 kr	Godkänt
Felhantering vid brutet API	Ett tydligt felmeddelande i röd text visas om nätverksanropet misslyckas	Godkänt
HTML-validering (Nu Html Checker)	Inga kritiska strukturfel i koden	Godkänt (2 mindre varningar upptäcktes)
CSS-validering (W3C)	Stilmallarna följer officiell webbstandard utan fel	Godkänt
Test på mindre och större skärmar	Layouten anpassas flytande via Flexbox och Grid utan horisontell scroll	Godkänt

7 Resultat

Om arbetet med applikationen fortsatte finns det flera områden som skulle kunna förbättras och vidare utvecklas för att höja användarupplevelsen och funktionaliteten:

- **Komplettering av designkrav:** prioritet i ett tidigt skede av fortsatte arbete skulle vara att färdigställa de delar från Figma-mockupen som exkluderats på grund av tidsbrist: filtreringsfunktionen, prenumerationssystemet samt overlay-fönstret fördjupad produktinformation.

- **Användarautentisering och datalagring:** För att applikationen ska bli ännu bättre i framtiden bör ett inloggningssystem introduceras. Genom att spara användarens information och inställningar lokalt via localStorage kan en mer personlig upplevelse skapas utan krav på tunga databaslösningar i ett första skede.

- **Anpassningsbart tema (Dark/Light mode):** En mörkt och ljust läge-funktion (Dark/Light mode) bör implementeras på själva applikationen. Detta ökar tillgängligheten och låter användaren anpassa gränssnittets estetisk utifrån eget önskemål och omgivnings ljusförhållanden.

8 Referenser

1. w3schools. Asynchronous Fetch API. [Online].; 2026 [cited 2026 oktober 30. Available from: https://www.w3schools.com/js/js_async_fetch.asp.
2. The super fast color palettes generator! COOLORS. [Online].; 2026 [cited 2026 oktober 30. Available from: https://coolors.co/.
3. The super fast color palettes generator. COOLORS. [Online].; 2026 [cited 2026 oktober 30. Available from: https://coolors.co/.
4. Canva. Canva LOGO. [Online].; 2026 [cited 2026 oktober 30. Available from: https://www.canva.com/.
5. Figma. Figma Help Center. [Online].; 2026 [cited 2026 oktober 30. Available from: https://www.figma.com/community.
6. Microsoft. Visula Studio Code. [Online].; 2026 [cited 2026 oktober 30. Available from: https://code.visualstudio.com/?_exp_home=e855cfac50.
7. Git Community. git-reset Documentation. [Online].; 2026 [cited 2026 oktober 30. Available from: https://git-scm.com/docs/git-reset.html#_description.
8. Community, Git. git-reflog Documentations. [Online].; 2026 [cited 2026 oktober 30. Available from: https://git-scm.com/search/results?search=reflog&language=en.
9. Git Community. git-cherry-pick Documentations. [Online].; 2026 [cited 2026 oktober 30. Available from: https://git-scm.com/search/results?search=reflog&language=en.
10. World Wide Web Consortium (W3C). W3C. [Online].; 2026 [cited 2026 oktober 30. Available from: https://jigsaw.w3.org/css-validator/#validate_by_input.
11. B F. colotToggle. 5th ed. Brimingham: Packt Publishing; 2024.
12. w3schools. JavaScript Array map(). [Online].; 2026 [cited 2026 oktober 30. Available from: https://www.w3schools.com/jsref/jsref_map.asp.
13. w3schools. The Spread. Operator. [Online].; 2026 [cited 2026 oktober 30. Available from: https://www.w3schools.com/jsref/jsref_oper_spread.asp.
14. w3schools. preventDefault() Event Method. [Online].; 2026 [cited 2026 oktober 30. Available from: https://www.w3schools.com/jsref/event_preventdefault.asp.

15. JavaScript String trim(). [Online].; 2026 [cited 2026 oktober 30. Available from: https://www.w3schools.com/jsref/jsref_trim_string.asp.

16. w3schools. HTML Iframe Syntax. [Online].; 2026 [cited 2026 oktober 30. Available from: https://www.w3schools.com/html/html_iframe.asp.

17. W3C. [Online].; 2026 [cited 2026 oktober fredag. Available from: https://validator.w3.org/.

18. w3chools. [Online].; 2026 [cited 2026 October 30. Available from: https://www.w3schools.com/css/css3_flexbox.asp.